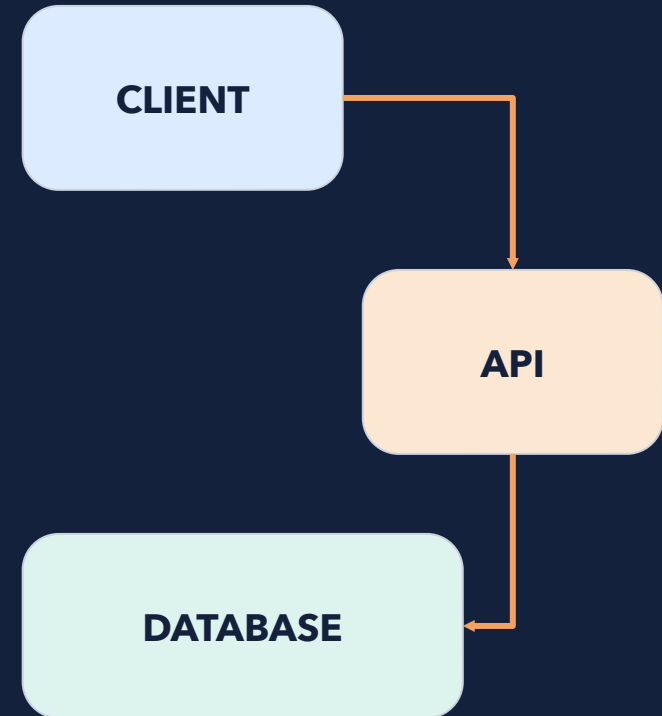


How Applications Use Databases

A saved request disappears from a dashboard. The stored rows explain why.



A repair request missing from the dashboard

One fictional repair desk will connect the application, the stored data, and the query.

Stored facts

Each ticket has an ID, a subject, a status, and an assignee.

One row represents one request.

Dashboard promise

Every active request should appear.

In this case, active means status new or open.

Reported problem

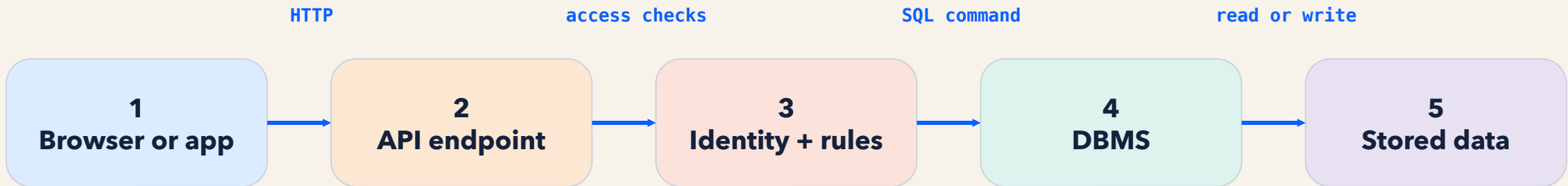
A student has confirmation 1004 for a broken bench.

Staff cannot find it on the dashboard.

A stored record and a query result can contain different rows.

Creating a support request

The application checks the request before asking the DBMS to store a row.



REQUEST

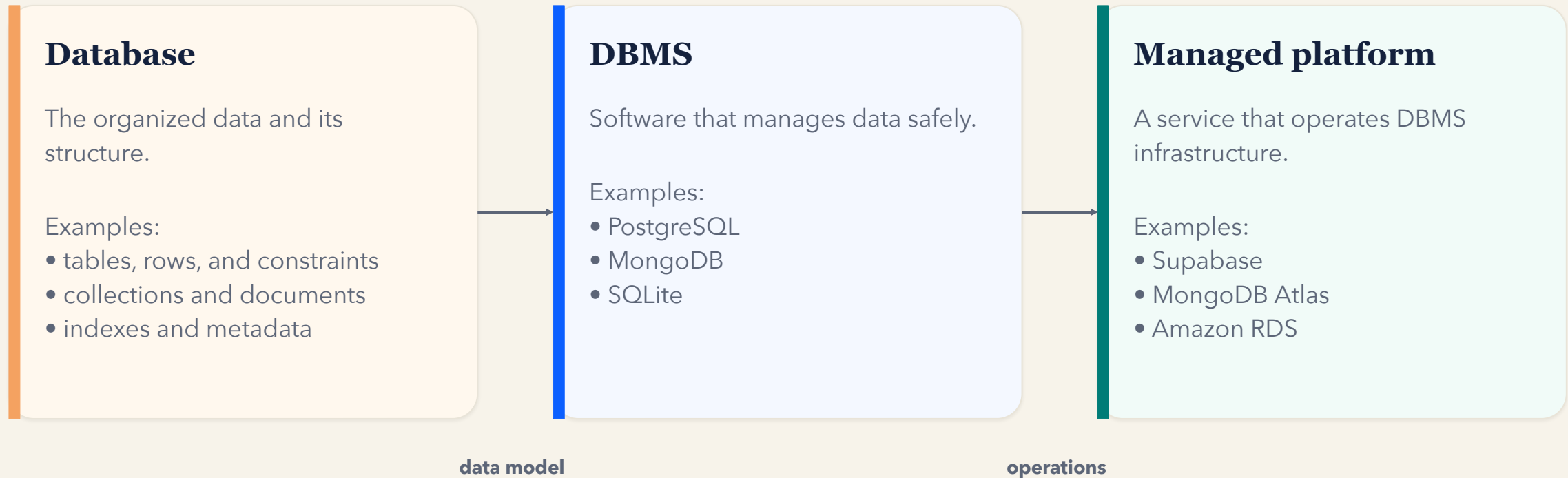
```
POST /api/tickets
{
  "subject": "Broken bench slat"
}
```

ILLUSTRATIVE SQL

```
INSERT INTO tickets
  (ticket_id, subject,
   status, assignee_id)
VALUES (1004, 'Broken bench slat',
       'new', NULL);
```

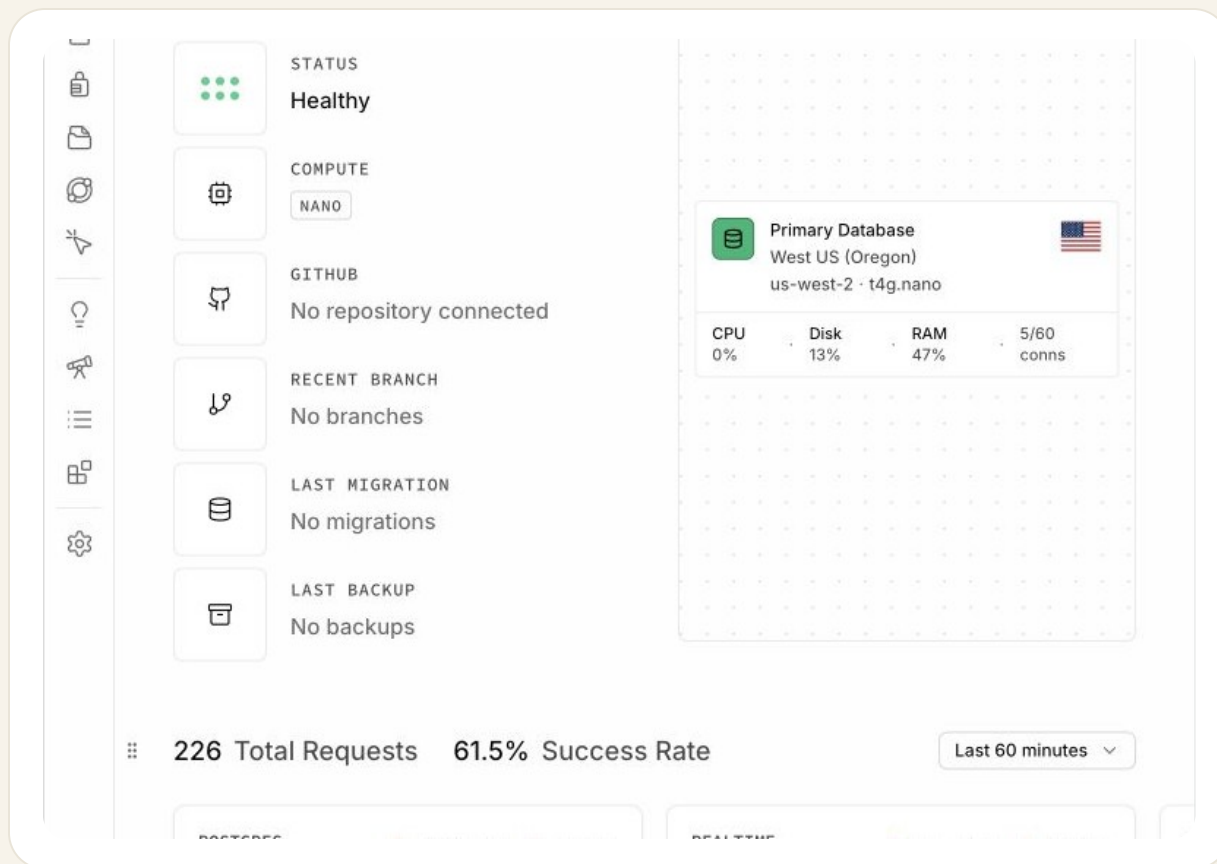
Database, DBMS, and managed platform

Separating the layers makes architecture and troubleshooting clearer.



Supabase is a platform built around PostgreSQL

The dashboard exposes projects and services; PostgreSQL remains the database engine.



What the platform adds

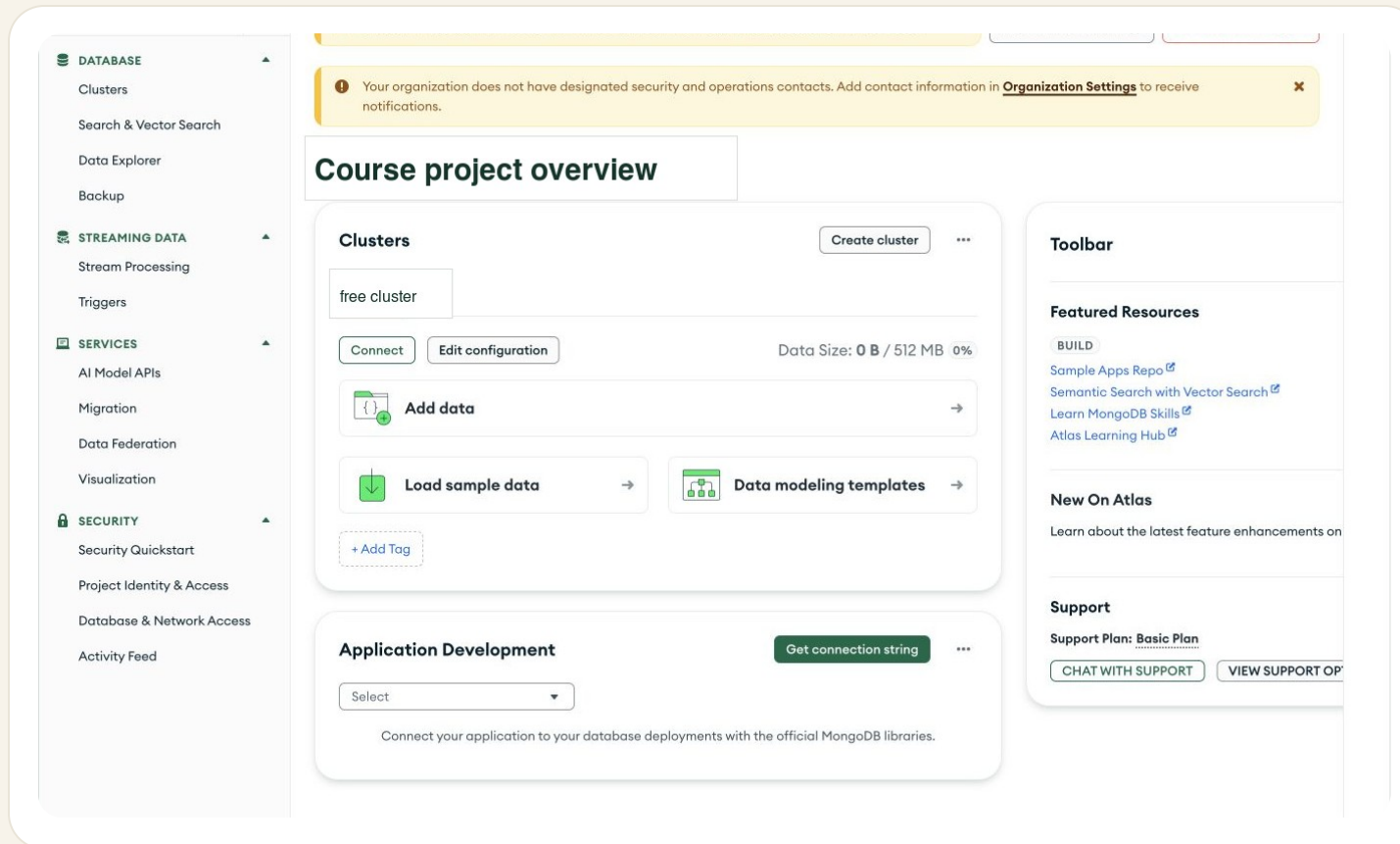
Dashboard · managed infrastructure · API services · authentication · logs · configuration

What PostgreSQL still does

Tables · SQL · constraints · transactions · query planning · indexes · roles

MongoDB Atlas operates MongoDB in the cloud

Projects, clusters, users, network rules, and data tools surround the document database.



Atlas layer

Project · cluster · network access ·
database users · logs

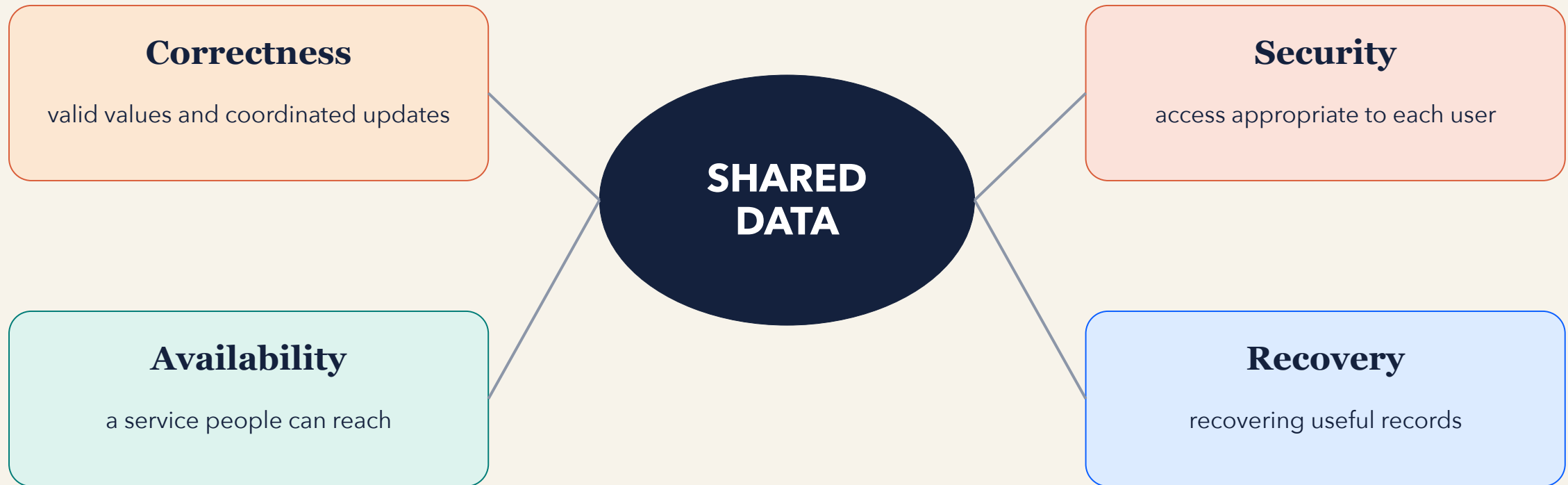
MongoDB layer

Databases · collections · BSON
documents · queries · indexes

Dated interface example. No cloud account is needed for this week's lab.

Responsibilities of database administration

The job is to keep shared data correct, available, secure, and recoverable.



A saved ticket missing from a report

The write path and the later read path need separate checks.

Observed symptom

The form reports success, but ticket 1004 is absent from the staff dashboard.

ONE OBSERVATION

```
HTTP 201 Created
{ "ticket_id": 1004,
  "status": "new" }
```

1

Client

Did the browser send the request?

2

API

What status and body came back?

3

Database

Did the command commit?

4

Read path

Is the later query filtering it out?

The Relational Model

A precise language for tables, questions, and transformations

**A row represents one fact.
Queries produce new results.**

RELATIONAL ALGEBRA

```
 $\sigma$  status = 'open' (tickets)  
 $\pi$  ticket_id, subject (tickets)
```

One row represents one support request

Three rows from the lab, with four attributes. Subjects are shortened for display.

ticket_id	subject	status	assignee_id
1001	Streetlight dark	open	201
1003	Low water pressure	resolved	201
1004	Broken bench slat	new	NULL

schema

attribute names + domains

tuple

one complete row of values

key

stable identity

Schema and current data

The schema describes allowed structure. The instance contains the rows present now.

Schema

```
tickets(  
  ticket_id integer,  
  subject text,  
  status text,  
  assignee_id integer  
)
```

Schema notation, not a SQL command.

Instance

The tuples stored at one moment.

At 10:00: 3 tickets

At 10:05: 4 tickets

Changes whenever rows are inserted, updated, or deleted.

Status values

new, open, resolved, closed

A domain constrains values. Nullability separately controls whether a value is required.

An assignee ID refers to an agent record

A nullable foreign key permits an unassigned ticket without inventing an agent.

agents

agent_id	name
201	Priya Shah
202	Noah Williams

tickets

ticket_id	subject	assignee_id
1001	Streetlight dark	201
1003	Low water pressure	201
1004	Broken bench slat	NULL



PK ← FK

Ticket 1001 refers to agent 201. Ticket 1004 has no assigned agent yet.

Selection filters rows

σ is sigma. This open-only condition keeps ticket 1001 in the displayed excerpt.

ticket_id	subject	status
1001	Streetlight dark	open
1003	Low water pressure	resolved
1004	Broken bench slat	new

 σ **ALGEBRA**

σ status = 'open' (tickets)

SQL

```
SELECT *  
FROM tickets  
WHERE status = 'open';
```

Projection keeps selected attributes

π is pi. The SQL output list chooses columns. DISTINCT explicitly removes duplicates.

ticket_id	subject	status	assignee_id
1001	Streetlight dark	open	201
1003	Low water pressure	resolved	201
1004	Broken bench slat	new	NULL

 π

ALGEBRA

 π ticket_id, subject (tickets)

SQL

```
SELECT ticket_id, subject
FROM tickets;
```

Output: ticket_id and subject

Products and joins

The full lab has four tickets and two agents. Matching IDs determine the join result.

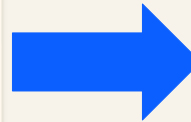
Cartesian product $R \times S$

Pairs every ticket with every agent.

$|tickets| = 4$ and $|agents| = 2$

$|tickets \times agents| = 8$

These include unrelated pairs.



condition

Equi-join $R \bowtie S$

An inner join keeps matching pairs.

$tickets.assignee_id$
 $= agents.agent_id$

A ticket with no matching agent produces no inner-join row.

A left join can preserve a ticket even when its assignee has no match.

The current dashboard query

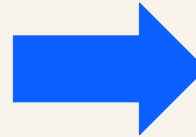
Active status and a matching agent are separate requirements in this query.

MEANING

Input: tickets and agents

Match the assignee ID to an agent ID.
Keep status new or open.
Return ticket ID, subject, and name.

An unassigned ticket has no match.



SQL

```
SELECT t.ticket_id, t.subject, a.name  
FROM tickets AS t  
JOIN agents AS a  
  ON t.assignee_id = a.agent_id  
WHERE t.status IN ('new', 'open');
```

1 match agent IDs

2 keep active status

3 choose output columns

Stored data and displayed results

Next week: SQL filters, NULL, joins, summaries, and safe practice changes.

Request path

client → API → identity → DBMS → data

System layers

database ≠ DBMS ≠ managed platform

Row meaning

relations, attributes, tuples, domains, keys

Query operations

σ selection · π projection · $\bowtie$ join

Before Week 2: review filtering, NULL, and joins. Reading links are in the weekly guide.